

Taller 1 - Sintaxis y herramientas

Sistemas Digitales

Segundo Cuatrimestre 2026

Guía de referencia de SystemVerilog y del flujo de simulación y síntesis.

1. Módulos, puertos y assign

Un módulo tiene interfaz (puertos) y cuerpo (lógica). Las entradas entran, las salidas se asignan, las señales internas conectan subexpresiones.

```
module mayor (  
    input logic a,  
    input logic b,  
    output logic y  
);  
    logic p;  
    assign p = a & b; // AND: 1 solo si ambos son 1  
    assign y = p | ~a; // OR y NOT  
endmodule
```

assign describe un cable: la salida sigue a la expresión en todo momento.

Lógica	SV	Nombre	Notas
$X \wedge Y$	$x \& y$	AND	1 solo si ambos son 1
$X \vee Y$	$x y$	OR	1 si alguno es 1
$\overline{X}$	$\sim x$	NOT	invierte el bit
$X \oplus Y$	$x \wedge y$	XOR	1 si los bits difieren

XOR se puede reescribir con AND, OR y NOT:

$$X \oplus Y = (X \wedge \overline{Y}) \vee (\overline{X} \wedge Y)$$

2. Buses

Un bit se declara `logic x`. Un grupo de bits, con rango:

```
logic [3:0] a; // 4 bits: a[3] a[2] a[1] a[0]
```

[3:0]: el índice alto a la izquierda [3] es bit más significativo o MSB; [0] es el bit menos significativo o LSB.

Literales con ancho y base:

```
4'b0111 // 4 bits en binario: literalmente los bits `0111`
```

```
1'b0 // un bit en 0
```

```
5'd1 // 5 bits en decimal: Los 5 bits que representan el 5 en decimal, que son  
00101
```

&, |, ^ y ~ también operan **bit a bit** sobre un bus: ~a invierte los 4 bits; a ^ b hace XOR de cada par a[i], b[i].

3. Concatenación y replicación

Llaves juntan bits. {3{en}} copia en tres veces: {en, en, en}.

```
assign word    = {hi, lo};      // dos buses (o bits) pegados
assign copies = {3{en}};       // {en, en, en}
```

4. Condicional (multiplexor)

sel ? a : b vale a si sel es 1, y b si es 0. Es un mux de 2 a 1: elige entre dos cables según un bit de control.

```
assign out = sel ? e_1 : e_0; // sel=1 → out=e_1; sel=0 → out=e_0
```

También opera sobre buses: ctrl ? resta : suma elige uno de los dos resultados.

5. Cómo conectar módulos

Instanciar es poner una **copia** de un módulo adentro de otro y cablear puertos. Cada copia tiene un nombre de instancia distinto.

Ejemplo: Dado el modulo xor2, que hace XOR de dos bits, así se implementaría el módulo paridad3, que verifica la paridad de tres entradas, y utiliza el modulo xor2 en su implementación.

```
module xor2 (
    input logic a,
    input logic b,
    output logic y
);
    assign y = a ^ b;
endmodule

module paridad3 (
    input logic x,
    input logic y,
    input logic z,
    output logic f
);
    logic p;

    xor2 u0 ( // llamaremos u0 a esta copia del modulo xor2
        .a(x), // al input a de u0, le conectamos x (el 1er input de paridad3)
        .b(y), // al input b de u0, le conectamos y (el 2do input de paridad3)
        .y(p)  // al output y de u0, lo conectamos nuestro auxiliar p (cableamos el
output de u0 a p)
    );

    xor2 u1 ( // llamaremos u1 a esta segunda copia del modulo xor2
        .a(p), // al input a de u1, le conectamos p, que tiene la salida de u0
        .b(z), // al input b de u1, le conectamos z (el 3er input de paridad3)
        .y(f)  // al output y de u1, lo conectamos a la salida f (salida del modulo
paridad3)
    );
endmodule
```

- xor2 es el tipo de módulo. u0 y u1 son los apodos de las dos copias.
- la sintaxis .a(x) conecta el puerto a del modulo que se esta instanciando con la señal x que vive dentro del modulo padre.
- logic p es un cable interno: sale de u0 y entra a u1.

Conexión por nombre, no posicional. Si se mezclan los puertos, el circuito compila y está mal.

6. Cómo traer un módulo de otro archivo

Instanciar usa el **nombre** del module. Ese module tiene que estar en algún .sv que el compilador vea.

No se editan copias a mano. El Makefile lista los .sv de ejercicios anteriores. `make sim` (o `make deps`) los **copia** a la carpeta actual, pisando copias viejas. La fuente de verdad es el ejercicio de origen: hay que resolver en orden.

HDL Studio no lee el Makefile y *Create circuit* suma solo el archivo abierto. Por eso la copia: los .sv quedan al lado del módulo nuevo. Hay que **agregarlos al circuito** (clic derecho → *Add to HDL Studio*, o seleccionar todos los .sv salvo el `_tb` y *Create circuit*). Sin ese paso, Yosys dice `unknown module`.

Si se quiere reutilizar un módulo propio extra: copiarlo a la carpeta (o sumarlo a `PRE_SRCS` del Makefile) y agregarlo al circuito de HDL Studio.

``include` pega un archivo en SystemVerilog. Verilator lo sigue. HDL Studio no: copia los fuentes a un temporal. Para sintetizar, el .sv tiene que estar **agregado al circuito**.

SystemVerilog también tiene `import` para **packages**. Acá los módulos se reutilizan por instancia + archivos en la carpeta.

7. Cómo correr la simulación

En la carpeta del ejercicio:

```
make sim
```

Si todos los casos pasan, aparece un recuadro verde **PASS**. Si alguno falla, líneas **FAIL: value = ...** y el proceso termina con error.

`make clean` borra `obj_dir/` (el compilado). No hace falta en el flujo normal: `make sim` vuelve a construir.

Los `_tb.sv` son parte de la especificación: se pueden leer, no se editan.

8. Cómo sintetizar

Simular comprueba comportamiento. Sintetizar muestra el **circuito**.

1. En la carpeta del ejercicio: `make sim` (copia los .sv anteriores si hace falta).
2. Abrir el .sv del módulo nuevo (no el `_tb`) y *Create circuit in HDL Studio*.
3. Agregar al circuito los .sv copiados que instancia (ahora están en la misma carpeta). No agregar el `_tb.sv`. Atajo: seleccionar todos los .sv menos el `_tb` y *Create circuit*.
4. **Top module**: el módulo nuevo. *Synthesize*.

Si el log dice `unknown module`, falta un .sv en el circuito.

9. Herramientas y bibliotecas

Nada de esto se instala a mano: vive en el contenedor (`twint/sd`). Esta sección es para tener de referencia, no se requiere aun comprender cada parte.

9.1. Entorno

- **VS Code + Dev Container**. `.devcontainer/devcontainer.json` pide la imagen y las extensiones. Al abrir la carpeta, “Reopen in Container”.

- **HDL Studio** (twint.hdl-studio). Extensión de la materia. Toma el SystemVerilog, lo sintetiza y dibuja el circuito interactivo.
- **Verilog HDL** (mshr-h.veriloghdl). Coloreo y navegación del código.

9.2. Simulación

- **Verilator** (verilator.org). Compila SystemVerilog a C++ y lo ejecuta. `make sim` llama `verilator --binary --timing con --top-module sim_top`.
- **Make**. Cada carpeta define TB, SRCS y a veces PRE_SRCS. `make sim` copia PRE_SRCS a la carpeta y después llama a Verilator.
- **sim_top.sv**. Genera `clk` y `rst`, instancia el testbench (TB_MODULE) y corta la corrida con `$fatal` si falló algún caso.
- **oracle_tb.sv**. Recorre los 2^N valores de entrada. El testbench concreto dice qué es un caso correcto (`pass`); el oráculo cuenta aciertos y pinta PASS/FAIL.

No hace falta escribir testbenches en este taller. Si se quiere profundizar: el oráculo es un módulo parametrizado (`#(parameter N = ...)`), el top usa un ``define` para elegir el TB, y Verilator traduce el testbench a C++.

9.3. Síntesis y lenguaje

- **Yosys** (yosyshq.net/yosys). Sintetizador de código abierto. HDL Studio lo usa (`read_slang`) para inferir compuertas a partir de los `assign` e instancias. Copia al directorio temporal solo los `.sv` agregados al circuito.
- **SystemVerilog (IEEE 1800)**. El subconjunto de este taller está en esta guía: `module`, `logic`, `assign`, operadores bit a bit, el condicional `?`, instancias, ``include` y la lista de archivos del `Makefile`. El estándar completo está en IEEE; una introducción amigable es Harris y Harris, **Digital Design and Computer Architecture** (capítulos de HDL combinacional).

9.4. En el contenedor, todavía no vistos en los talleres

- **Slang**. Language server: subrayado de errores mientras se escribe. El frontend `read_slang` de Yosys usa el mismo compilador.
- **Verible**. Formateador y linter (`verible-verilog-format`).
- **Surfer**. Visor de formas de onda. Útil cuando la simulación genera un dump (FST/VCD); este taller solo imprime PASS/FAIL.